

UNIVERSIDAD TECNOLOGICA DE MEXICO

GRUPO: TG0756

MATERIA: MICROPROCESADORES



TITULO: PROYECTO DUAL

NOMBRE DEL ALUMNO: JORGE FABIAN AVILA RODRIGUEZ

NOMBRE DEL PROFESOR QUE IMPARTE LA MATERIA: JORGE JAVIER
PEREZ HEREDIA

Proyecto Integrador Dual

MediCita: Sistema automático de citas médicas

1. Introducción

El presente proyecto tiene como finalidad desarrollar un sistema web inteligente para el agendamiento automático de citas médicas en un entorno hospitalario. La propuesta surge a partir de la necesidad de optimizar los procesos administrativos relacionados con la programación de consultas, ya que en muchos hospitales o clínicas el registro de citas todavía se realiza de manera manual, mediante llamadas telefónicas, hojas de cálculo o libretas.

Este tipo de registro puede generar pérdida de tiempo, errores de captura, empalmes de horarios y dificultad para consultar la disponibilidad de médicos, especialidades y consultorios. Por ello, el proyecto propone el desarrollo de un Producto Mínimo Viable, también conocido como MVP, que permita registrar pacientes, médicos, especialidades, consultorios y citas, incorporando un módulo automático de sugerencia de horarios disponibles.

El sistema no solo buscará facilitar el registro de citas, sino también generar datos que permitan medir su eficiencia. Para ello, se integrará una función de registro de logs, la cual almacenará información sobre el tiempo de agendamiento, intentos fallidos, errores de empalme y citas registradas correctamente. Estos datos permitirán validar si la solución propuesta realmente mejora el proceso tradicional.

2. Contexto de la organización

Para el desarrollo del proyecto se plantea como caso de estudio al Hospital VidaPlena, una institución médica ficticia dedicada a brindar atención en distintas especialidades, tales como medicina general, pediatría, ginecología, cardiología y traumatología.

Actualmente, el proceso de agendamiento de citas se realiza de forma manual por parte del personal administrativo. La recepcionista debe revisar la disponibilidad del médico, confirmar el horario solicitado por el paciente, verificar que no exista otra cita registrada en el mismo espacio y posteriormente anotar la información en una hoja de cálculo o registro físico.

Este método puede funcionar en escenarios de baja demanda, pero cuando aumenta el número de pacientes, médicos y especialidades, el proceso se vuelve lento y

propenso a errores. Por esta razón, se propone implementar una solución tecnológica que automatice parte del proceso y ayude a mejorar la eficiencia del área administrativa.

3. Marco teórico

3.1 Sistemas de información en el área médica

Los sistemas de información en el área médica permiten organizar, almacenar, consultar y administrar datos relacionados con pacientes, médicos, servicios, horarios y procesos internos de una institución de salud. Su uso contribuye a mejorar la eficiencia administrativa, reducir errores en el manejo de información y facilitar la toma de decisiones dentro de hospitales o clínicas.

En el caso del proyecto MediCI, el sistema se enfoca en el proceso de agendamiento de citas médicas, ya que este representa una actividad frecuente dentro de una institución hospitalaria. Cuando este proceso se realiza de forma manual, puede generar retrasos, duplicidad de registros, empalmes de horarios y dificultad para consultar la disponibilidad de médicos y consultorios.

Por esta razón, el desarrollo de un sistema web de citas médicas permite automatizar parte del proceso administrativo y generar información útil para evaluar su desempeño.

3.2 Gestión de citas médicas

La gestión de citas médicas consiste en organizar la asignación de pacientes con médicos, especialidades, consultorios, fechas y horarios disponibles. Este proceso debe considerar diferentes elementos para evitar errores, como la disponibilidad del médico, la duración de la consulta, el consultorio asignado y el estado de la cita.

Un sistema de citas médicas debe permitir registrar, consultar, cancelar y reprogramar citas. Además, debe validar que un médico o consultorio no tenga más de una cita asignada en el mismo horario. Esta validación es importante porque evita empalmes y mejora la organización de la agenda hospitalaria.

En MediCI, la gestión de citas será apoyada por un módulo de asignación automática, el cual revisará la disponibilidad registrada en la base de datos y sugerirá el horario más cercano disponible.

3.3 Automatización de procesos

La automatización de procesos consiste en utilizar software para realizar tareas repetitivas o de validación que normalmente se hacen de manera manual. En un entorno hospitalario, automatizar el agendamiento de citas puede disminuir el tiempo que tarda el personal administrativo en registrar una consulta y reducir errores provocados por revisión manual.

En este proyecto, la automatización se aplicará mediante un algoritmo de asignación de horarios. El sistema analizará los médicos disponibles, sus horarios de atención, los consultorios libres y las citas ya programadas para determinar si un espacio puede ser ocupado. Si el horario solicitado no está disponible, el sistema podrá sugerir otra opción cercana.

Esta funcionalidad permite que el proyecto no se limite a un CRUD básico, ya que incorpora una lógica de validación y recomendación de horarios.

3.4 Arquitectura cliente-servidor

La arquitectura cliente-servidor es un modelo de desarrollo en el que una aplicación se divide en dos partes principales. El cliente corresponde a la interfaz que utiliza el usuario, mientras que el servidor procesa las solicitudes, aplica reglas de negocio y se comunica con la base de datos.

Para MediCI se propone una arquitectura cliente-servidor dividida en frontend, backend y base de datos. El frontend será la interfaz del usuario, desarrollada con React. El backend será una API desarrollada con Node.js y Express, encargada de recibir solicitudes, validar datos, ejecutar reglas de negocio y conectarse con MySQL. La base de datos almacenará la información de pacientes, médicos, especialidades, consultorios, citas y logs.

Esta separación permite mantener una estructura ordenada y facilita el mantenimiento del sistema.

3.5 API REST

Una API REST permite la comunicación entre el frontend y el backend mediante solicitudes HTTP. En este proyecto, el frontend enviará solicitudes al backend para registrar pacientes, consultar médicos, crear citas, validar horarios y guardar logs de agendamiento.

El uso de una API REST permite que la lógica principal del sistema no esté mezclada con la interfaz visual. Esto favorece la separación de responsabilidades y permite que el sistema sea más escalable y fácil de mantener.

Endpoints propuestos para MediCI

Método	Endpoint	Función
GET	/api/health	Verificar que la API funcione
GET	/api/pacientes	Consultar pacientes
POST	/api/pacientes	Registrar pacientes
GET	/api/especialidades	Consultar especialidades médicas
POST	/api/especialidades	Registrar especialidades
GET	/api/medicos	Consultar médicos
POST	/api/medicos	Registrar médicos
GET	/api/consultorios	Consultar consultorios
POST	/api/consultorios	Registrar consultorios
GET	/api/citas	Consultar citas
POST	/api/citas	Crear cita médica
POST	/api/citas/sugerir-horario	Sugerir horario disponible
POST	/api/logs	Guardar log de agendamiento

3.6 Base de datos relacional

Una base de datos relacional permite organizar la información en tablas relacionadas mediante llaves primarias y llaves foráneas. Para MediCI se utilizará MySQL, ya que permite almacenar información estructurada y mantener relaciones entre pacientes, médicos, especialidades, consultorios, citas y logs.

El uso de una base de datos relacional es adecuado para este proyecto porque el proceso de citas médicas requiere mantener relaciones claras. Por ejemplo, una cita pertenece a un paciente, se asigna a un médico, se realiza en un consultorio y queda registrada por un usuario del sistema.

Además, la base de datos permitirá almacenar los logs necesarios para validar la hipótesis del proyecto.

3.7 Logs e instrumentación del software

La instrumentación del software consiste en programar el sistema para generar datos sobre su propio funcionamiento. En MediCI, esta instrumentación se realizará mediante una tabla de logs de citas.

Cada vez que se intente registrar una cita, el sistema deberá guardar información como el método de agendamiento, el tiempo de registro, si hubo error de empalme, si la cita fue exitosa, si el sistema sugirió un horario y cuántos intentos fueron necesarios.

Estos datos permitirán comparar el método manual simulado contra el método automático. De esta forma, el proyecto podrá comprobar si la implementación del sistema reduce el tiempo promedio de registro y disminuye los errores de empalme.

3.8 Seguridad básica en aplicaciones web

La seguridad básica en una aplicación web incluye prácticas como proteger contraseñas, validar datos, evitar credenciales expuestas en el código fuente y controlar el acceso de usuarios. Para MediCI se considera el uso de variables de entorno para almacenar credenciales de base de datos, además de una estructura de login para usuarios administradores o recepcionistas.

También se deberán implementar validaciones para evitar registros incompletos, datos inválidos o acciones no permitidas dentro del sistema. Estas medidas ayudan a mejorar la calidad del software y reducen errores durante el uso del sistema.

3.9 Estado del arte

Actualmente existen soluciones digitales para administrar citas, agendas y procesos médicos. Algunas plataformas permiten registrar pacientes, consultar calendarios y organizar servicios de salud. Sin embargo, muchas funcionan principalmente como herramientas de agenda o registro, sin enfocarse en medir el impacto del proceso de agendamiento.

La diferencia principal de MediCI es que no solo busca registrar citas médicas, sino también automatizar la sugerencia de horarios disponibles y medir el desempeño del proceso mediante logs generados por el sistema. Esto permite que el proyecto tenga una doble finalidad: desarrollar una solución funcional y validar con datos si la automatización reduce tiempos y errores.

MediCI integra tres elementos principales:

1. Gestión de citas médicas.

2. Asignación automática de horarios disponibles.
3. Registro de métricas para validar una hipótesis.

Por esta razón, el proyecto va más allá de un sistema básico de registro, ya que incluye una lógica de automatización y un enfoque de investigación aplicada.

3.10 Fundamentación académica complementaria

Como parte de la retroalimentación recibida en la Bitácora 1, se identificó la necesidad de complementar el proyecto con fuentes académicas indexadas y no depender únicamente de documentación técnica oficial. Por ello, se incorporan referencias relacionadas con sistemas web de citas médicas, programación de citas en servicios de salud e interoperabilidad mediante estándares como HL7 FHIR.

La revisión sistemática de Zhao, Yoo, Lavoie, Lavoie y Simoes sobre sistemas web de citas médicas permite justificar la importancia de implementar plataformas digitales para gestionar agendamientos en el área de salud. Asimismo, el estudio de Ala, Alsaadi, Ahmadi y Mirjalili sobre programación de citas en servicios de salud permite sustentar la relevancia de automatizar procesos de asignación, disponibilidad y reducción de errores.

Además, HL7 FHIR Appointment se considera como referencia técnica para comprender cómo los sistemas de salud modelan el concepto de una cita médica dentro de estándares de interoperabilidad.

4. Planteamiento del problema

El proceso manual de agendamiento de citas médicas genera tiempos elevados de registro, errores en la asignación de horarios y posibles empalmes entre pacientes, médicos y consultorios.

Esta situación afecta la organización interna del hospital, ya que el personal administrativo debe revisar manualmente la disponibilidad de cada médico y confirmar que el horario solicitado no esté ocupado. Cuando este proceso se realiza sin apoyo tecnológico, pueden presentarse errores como citas duplicadas, datos incompletos, asignaciones incorrectas o pérdida de tiempo al buscar espacios disponibles.

Además, la falta de datos históricos dificulta conocer cuántas citas se registran correctamente, cuántos intentos fallan por conflicto de horario y cuánto tiempo tarda realmente el proceso de agendamiento.

Por lo anterior, se identifica la necesidad de desarrollar un sistema web que permita automatizar la asignación de citas médicas, evitar empalmes y generar datos medibles sobre el desempeño del proceso.

5. Justificación

El desarrollo de un sistema automático de citas médicas es importante porque permite mejorar la eficiencia administrativa en hospitales y clínicas. La automatización del proceso puede ayudar a reducir el tiempo que tarda el personal en registrar una cita, disminuir errores de programación y facilitar la consulta de horarios disponibles.

Desde el punto de vista tecnológico, el proyecto permite aplicar conocimientos de Ingeniería en Sistemas Computacionales, tales como diseño de bases de datos, desarrollo web, arquitectura cliente-servidor, validación de datos, lógica de negocio, manejo de errores, seguridad básica e instrumentación del software mediante logs.

Además, el proyecto tiene valor investigativo, ya que no solo se desarrollará una aplicación funcional, sino que también se medirá su impacto mediante datos generados por el propio sistema. De esta manera, será posible comparar el proceso manual con el proceso automatizado y determinar si la solución propuesta reduce tiempos y errores.

6. Objetivo general

Desarrollar un sistema web inteligente de agendamiento automático de citas médicas para optimizar el registro de consultas, reducir tiempos administrativos y disminuir errores de empalme de horarios en un entorno hospitalario.

7. Objetivos específicos

1. Diseñar una base de datos que permita registrar pacientes, médicos, especialidades, consultorios, horarios y citas médicas.
2. Implementar un módulo de asignación automática que sugiera horarios disponibles de acuerdo con la especialidad, médico, fecha y consultorio.
3. Desarrollar una validación que evite el registro de dos citas en el mismo horario, con el mismo médico o consultorio.

- 4. Integrar un módulo de logs que registre el tiempo de agendamiento, errores detectados, intentos fallidos y citas exitosas.
- 5. Comparar el proceso manual simulado contra el proceso automatizado mediante métricas generadas por el sistema.
- 6. Validar la hipótesis mediante el análisis de los datos recolectados durante las pruebas del MVP.

8. Hipótesis causal

La implementación de un sistema automático de asignación de citas médicas reducirá en un 40% el tiempo promedio de registro de una cita y disminuirá en un 50% los errores de empalme de horarios en comparación con el método manual tradicional.

9. Variables de investigación

Tipo de variable	Variable
Variable independiente	Sistema automático de asignación de citas médicas
Variable dependiente 1	Tiempo promedio de registro de una cita
Variable dependiente 2	Errores de empalme de horarios
Variable dependiente 3	Citas registradas correctamente
Variable dependiente 4	Intentos fallidos de agendamiento

10. Métricas a medir

Métrica	Descripción	Unidad
Tiempo de registro	Tiempo que tarda el usuario en registrar una cita	Segundos
Errores de empalme	Intentos de registrar citas en horarios ocupados	Número de errores

Citas exitosas	Citas registradas correctamente	Número de citas
Intentos fallidos	Procesos no completados por conflicto o error	Número de intentos
Horarios sugeridos	Veces que el sistema propone un horario disponible	Número de sugerencias

11. Metodología

11.1 Diseño de la investigación

El proyecto se desarrollará bajo un enfoque de investigación aplicada, ya que busca resolver un problema práctico mediante el desarrollo de una solución tecnológica. La investigación se centrará en comparar el proceso manual simulado de agendamiento de citas contra el proceso automatizado propuesto en MediCI.

11.2 Población de prueba

La población de prueba estará compuesta por registros simulados de pacientes, médicos, consultorios y citas médicas. No se utilizarán datos reales de pacientes. Los datos serán ficticios y generados únicamente para fines académicos y de validación del sistema.

11.3 Muestra inicial

Para la fase final del proyecto se buscará generar al menos 20 registros reales dentro del sistema, considerando intentos de agendamiento manual simulado y automático. Cada registro deberá guardar métricas como tiempo de registro, existencia de errores, estado de la cita y método utilizado.

11.4 Diseño experimental preliminar

Para validar la hipótesis, se realizará una comparación entre dos métodos de agendamiento de citas médicas.

Método	Descripción
Manual simulado	El usuario revisa manualmente la disponibilidad del médico y consultorio antes de registrar la cita

Automático	El sistema valida disponibilidad y sugiere el horario disponible más cercano
------------	--

El primer método será el proceso manual simulado, en el cual el usuario deberá revisar la disponibilidad del médico y seleccionar un horario sin ayuda automática del sistema. Este método permitirá obtener una referencia inicial sobre el tiempo que tarda el proceso y los errores que pueden generarse.

El segundo método será el proceso automatizado, en el cual el sistema sugerirá horarios disponibles, validará la disponibilidad del médico y consultorio, y evitará empalmes de citas.

Durante las pruebas se registrarán al menos 20 eventos generados por el sistema. Cada evento guardará información como el método utilizado, tiempo de registro, resultado de la operación, existencia de errores de empalme y fecha de registro.

Posteriormente, se realizará una comparación entre los resultados obtenidos en el método manual y el método automatizado para determinar si el sistema reduce el tiempo promedio de registro y disminuye los errores de empalme.

12. Diseño de la solución

12.1 Arquitectura propuesta

MediCI utilizará una arquitectura cliente-servidor dividida en tres capas principales:

1. Frontend.
2. Backend.
3. Base de datos.

El frontend será desarrollado con React y permitirá al usuario interactuar con el sistema. El backend será desarrollado con Node.js y Express, y se encargará de procesar las solicitudes, aplicar validaciones y comunicarse con la base de datos. La base de datos MySQL almacenará la información del sistema.

12.2 Diagrama de arquitectura

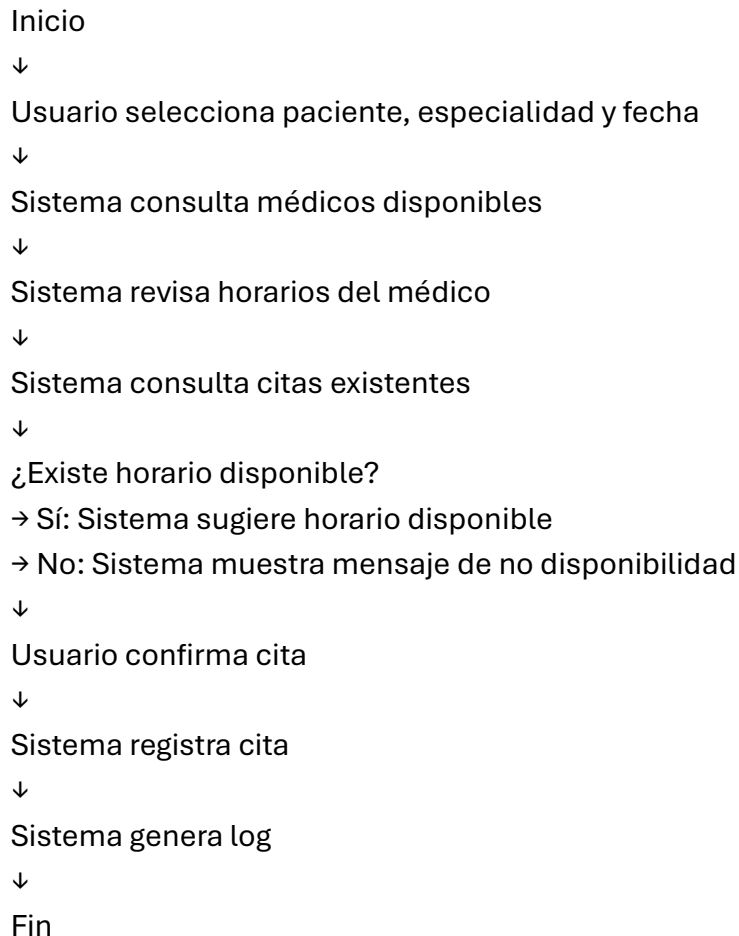
[Insertar imagen: Diagrama de Arquitectura - MediCI]

12.3 Explicación de la arquitectura

El usuario interactúa con el sistema mediante una interfaz web. Las acciones realizadas en el frontend generan solicitudes HTTP hacia la API REST. El backend recibe estas solicitudes, valida la información y ejecuta la lógica correspondiente.

Cuando se registra una cita, el backend verifica la disponibilidad del médico y consultorio. Si existe disponibilidad, guarda la cita en la base de datos. Si hay conflicto, evita el registro y puede sugerir otro horario. Además, cada intento genera un log para medir el desempeño del proceso.

12.4 Diagrama de flujo del agendamiento automático



12.5 Diseño de base de datos

La base de datos de MediCI estará compuesta por las siguientes tablas:

Tabla	Descripción
usuarios	Almacena usuarios administradores y recepcionistas

pacientes	Registra datos básicos de pacientes ficticios
especialidades	Catálogo de especialidades médicas
medicos	Información de médicos y su especialidad
consultorios	Espacios físicos donde se atienden citas
horarios_medicos	Días y horarios de atención de cada médico
citas	Registro principal de citas médicas
logs_citas	Registros generados para medir el desempeño del sistema

12.6 Relaciones principales

Relación	Descripción
especialidades - medicos	Una especialidad puede tener varios médicos
medicos - horarios_medicos	Un médico puede tener varios horarios disponibles
pacientes - citas	Un paciente puede tener varias citas
medicos - citas	Un médico puede atender varias citas
consultorios - citas	Un consultorio puede tener varias citas
usuarios - citas	Un usuario registra citas en el sistema
citas - logs_citas	Una cita puede tener registros de medición asociados

12.7 Diagrama entidad-relación

[Insertar imagen: Diagrama Entidad-Relación - MediCI]

12.8 Diagrama entidad-relación textual

ESPECIALIDADES 1 — N MEDICOS
 MEDICOS 1 — N HORARIOS_MEDICOS
 PACIENTES 1 — N CITAS
 MEDICOS 1 — N CITAS
 CONSULTORIOS 1 — N CITAS
 USUARIOS 1 — N CITAS
 CITAS 1 — N LOGS_CITAS

13. Stack tecnológico

Componente	Tecnología
Frontend	React
Backend	Node.js con Express
Base de datos	MySQL
Control de versiones	GitHub
Editor de código	Visual Studio Code
Diagramas	Draw.io
Análisis de datos	Excel o Python
Despliegue	Render, Railway o servidor local configurado

14. Requisitos funcionales

ID Requisito funcional

RF01 El sistema permitirá iniciar sesión al personal autorizado.

RF02 El sistema permitirá registrar pacientes.

RF03 El sistema permitirá consultar pacientes registrados.

RF04 El sistema permitirá registrar médicos y especialidades.

RF05 El sistema permitirá registrar consultorios.

RF06 El sistema permitirá registrar citas médicas.

RF07 El sistema verificará la disponibilidad del médico antes de guardar una cita.

RF08 El sistema verificará la disponibilidad del consultorio antes de guardar una cita.

RF09 El sistema evitará empalmes de citas en el mismo horario.

RF10 El sistema sugerirá horarios disponibles cuando el horario solicitado no esté libre.

RF11 El sistema permitirá cancelar o reprogramar citas.

RF12 El sistema registrará logs de cada intento de agendamiento.

RF13 El sistema mostrará métricas básicas de tiempo de registro, errores y citas exitosas.

15. Requisitos no funcionales

ID Requisito no funcional

RNF01 La interfaz deberá ser clara, intuitiva y responsiva.

RNF02 El backend deberá estar separado del frontend mediante una API REST.

RNF03 La base de datos deberá conservar la información de manera persistente.

RNF04 Las credenciales de base de datos deberán almacenarse en variables de entorno.

RNF05 El sistema deberá validar campos obligatorios antes de guardar información.

RNF06 El sistema deberá mostrar mensajes amigables ante errores de usuario o sistema.

RNF07 El código deberá organizarse por rutas, controladores, modelos y configuración.

RNF08 Las operaciones principales deberán responder en un tiempo aceptable para el usuario.

RNF09 El sistema deberá utilizar datos ficticios o anonimizados durante las pruebas.

RNF10 El repositorio deberá mantener commits constantes y descriptivos.

16. Descripción del MVP

El Producto Mínimo Viable será una aplicación web para la gestión y asignación automática de citas médicas. El sistema permitirá que una recepcionista o administrador registre pacientes, médicos, especialidades, consultorios y citas médicas.

El módulo principal será la asignación automática de horarios. Este módulo revisará la disponibilidad de los médicos, consultorios y horarios para sugerir al usuario el espacio más cercano disponible. Además, el sistema impedirá que se registren citas duplicadas o empalmadas.

El MVP también incluirá un módulo de logs, el cual permitirá guardar datos sobre cada intento de agendamiento. Estos datos serán utilizados para la validación de la hipótesis del proyecto.

17. Módulos principales del sistema

Módulo	Función
Login	Acceso de recepcionista o administrador
Pacientes	Registro y consulta de pacientes
Médicos	Registro y consulta de médicos
Especialidades	Registro y consulta de especialidades médicas
Consultorios	Registro y consulta de espacios disponibles
Citas	Crear, consultar, cancelar y reprogramar citas
Asignación automática	Sugerir horarios disponibles
Validación de empalmes	Evitar citas duplicadas
Logs	Guardar tiempos, errores e intentos
Dashboard	Mostrar citas y métricas básicas

18. Backlog de historias de usuario

ID	Historia de usuario
----	---------------------

- HU01 Como recepcionista, quiero iniciar sesión para acceder al sistema.
- HU02 Como recepcionista, quiero registrar pacientes.
- HU03 Como administrador, quiero registrar médicos y especialidades.
- HU04 Como recepcionista, quiero registrar una cita médica.
- HU05 Como sistema, quiero validar horarios ocupados.
- HU06 Como sistema, quiero sugerir automáticamente el horario disponible más cercano.
- HU07 Como recepcionista, quiero consultar las citas del día.
- HU08 Como administrador, quiero visualizar métricas del sistema.
- HU09 Como sistema, quiero guardar logs de cada intento de agendamiento.
- HU10 Como recepcionista, quiero cancelar o reprogramar citas.
-

19. Cronograma actualizado de trabajo

Semana	Actividad
Semana 4	Definición del proyecto, planteamiento del problema, hipótesis, métricas, creación del repositorio, README inicial y backlog.
Semana 5	Diseño y mejora de base de datos, estructura inicial del backend y definición de rutas principales.
Semana 6	Desarrollo del MVP esqueleto, conexión frontend-backend, diagramas, requisitos funcionales y no funcionales.
Semana 7	Atención a retroalimentación de Bitácora 1, actualización de referencias y desarrollo inicial del módulo de pacientes.
Semana 8	Desarrollo de módulos de especialidades, médicos y consultorios; actualización del frontend con dashboard inicial.
Semana 9	Desarrollo del módulo de citas médicas, validación de disponibilidad y prevención de empalmes.

Semana 10	Implementación de asignación automática, registro de logs, pruebas del sistema y generación de registros.
Semana 11	Análisis de resultados, conclusiones, documentación final y demo del MVP.

20. Repositorio

El repositorio del proyecto se alojará en GitHub para llevar el control de versiones y evidenciar el avance constante del desarrollo.

Link del repositorio:

<https://github.com/JorgeFabianAR/medici-sistema-citas-medicas>

21. Avance Semana 6: MVP esqueleto y conexión frontend-backend

Durante la Semana 6 se desarrolló el MVP esqueleto del sistema MediCI. Esta etapa permitió comprobar que el proyecto ya cuenta con una estructura funcional inicial, aunque todavía no incluye todas las funcionalidades finales.

En esta semana se configuró el backend con Node.js y Express, incluyendo el endpoint `/api/health`, el cual permite verificar que la API funciona correctamente. También se configuró el frontend con React, mostrando una pantalla inicial del sistema MediCI y validando la conexión con el backend.

21.1 Evidencia de conexión estable entre frontend y backend

[Insertar captura: frontend mostrando “Estado de conexión con API: API de MediCI funcionando correctamente”]

[Insertar captura: navegador mostrando respuesta JSON de `/api/health`]

Con esta evidencia se demuestra que el backend y el frontend pueden comunicarse mediante solicitudes HTTP, lo cual cumple con el objetivo técnico de Semana 6.

22. Avance Semana 7: Atención a retroalimentación y módulo de pacientes

Durante la Semana 7 se atendió la retroalimentación recibida en la Bitácora 1 del Proyecto Integrador Dual. Las observaciones principales estuvieron relacionadas con la necesidad de complementar las referencias con fuentes académicas indexadas,

corregir referencias registradas como “s.f.” y mantener evidencia constante de commits semanales en el repositorio de GitHub.

Como acción de mejora, se documentó en el repositorio la atención a la retroalimentación recibida, dejando evidencia mediante el archivo `retroalimentacion_bitacora1.md`. Asimismo, se inició el desarrollo funcional del sistema MediCI mediante la creación del módulo de pacientes en el backend.

En esta etapa se agregaron las carpetas controllers y routes dentro del backend, con el objetivo de mantener una estructura más ordenada y cercana a una arquitectura profesional. El módulo de pacientes incluye un controlador y una ruta específica para consultar pacientes de prueba.

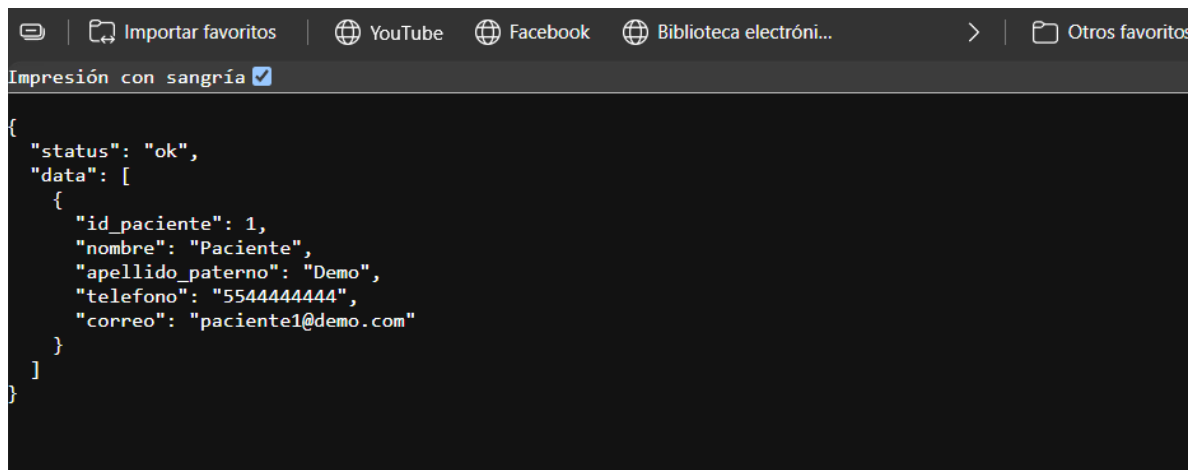
22.1 Endpoint implementado en Semana 7

Método	Endpoint	Función
--------	----------	---------

GET	/api/pacientes	Consultar pacientes registrados en el sistema
-----	----------------	---

Este endpoint permite comprobar que el backend ya no solo responde con una ruta de prueba, sino que comienza a manejar módulos funcionales del sistema.

22.2 Evidencia funcional de Semana 7



The screenshot shows a web browser window with a dark theme. The address bar displays 'Impresión con sangría' with a checked checkbox. The main content area shows a JSON response from the /api/pacientes endpoint. The JSON is formatted with syntax highlighting and includes a status field set to 'ok' and a data array containing one patient object. The patient object has fields for id_paciente (1), nombre (Paciente), apellido_paterno (Demo), telefono (5544444444), and correo (paciente1@demo.com).

```
{
  "status": "ok",
  "data": [
    {
      "id_paciente": 1,
      "nombre": "Paciente",
      "apellido_paterno": "Demo",
      "telefono": "5544444444",
      "correo": "paciente1@demo.com"
    }
  ]
}
```

La evidencia de Semana 7 corresponde a la ejecución del endpoint `/api/pacientes`, el cual devuelve una respuesta en formato JSON con información de pacientes ficticios. Esta evidencia demuestra el avance del MVP hacia una aplicación funcional por módulos.

22.3 Commits realizados en Semana 7

Durante esta semana se realizaron commits correspondientes a la atención de retroalimentación, actualización de referencias y desarrollo inicial del módulo funcional de pacientes, manteniendo la trazabilidad del avance semanal en GitHub.

 backend	Semana 7 - iniciar modulo funcional d...	19 hours ago
 database	Update schema.sql	3 weeks ago
 docs	Semana 7 - actualizar referencias acad...	3 days ago
 frontend	Agregar evidencias de funcionamiento...	2 weeks ago
 .gitignore	Initial commit	last month
 README.md	Semana 7 - actualizar README con av...	19 hours ago

23. Avance Semana 8: Módulos de especialidades, médicos, consultorios y dashboard inicial

Durante la Semana 8 se continuó con el desarrollo funcional del sistema MediCI, ampliando el backend mediante la creación de nuevos módulos para especialidades, médicos y consultorios. Estos módulos son necesarios para que posteriormente el sistema pueda realizar el proceso de agendamiento de citas médicas con validación de disponibilidad.

En el backend se agregaron controladores y rutas independientes para cada entidad, permitiendo mantener una separación clara de responsabilidades dentro del código. Esta estructura facilita el mantenimiento del sistema y permite que las funcionalidades puedan crecer de manera ordenada en las siguientes semanas.

23.1 Módulos implementados en Semana 8

Módulo	Descripción
Especialidades	Permite consultar las especialidades médicas disponibles, como medicina general, pediatría y cardiología.
Médicos	Permite consultar médicos registrados, incluyendo su especialidad, cédula profesional, correo y teléfono.

Consultorios	Permite consultar los espacios físicos disponibles para la atención médica.
Dashboard inicial	Muestra en el frontend el total de pacientes, especialidades, médicos y consultorios registrados.

23.2 Endpoints implementados en Semana 8

Método	Endpoint	Función
GET	/api/especialidades	Consultar especialidades médicas
POST	/api/especialidades	Registrar una especialidad
GET	/api/medicos	Consultar médicos registrados
POST	/api/medicos	Registrar un médico
GET	/api/consultorios	Consultar consultorios disponibles
POST	/api/consultorios	Registrar un consultorio

23.3 Actualización del frontend

El frontend fue actualizado para consumir información proveniente de la API. La pantalla principal de MediCI ahora muestra un dashboard inicial con el conteo de pacientes, especialidades, médicos y consultorios. Además, muestra listas básicas de especialidades disponibles, médicos registrados y consultorios.

Esta actualización permite demostrar la comunicación entre el frontend desarrollado en React y el backend desarrollado con Node.js y Express. A diferencia de la Semana 6, donde únicamente se verificaba la conexión general con la API mediante /api/health, en Semana 8 el frontend ya consulta distintos módulos funcionales del sistema.

23.4 Evidencia funcional de Semana 8

Impresión con sangría ☒

```
{
  "status": "ok",
  "data": [
    {
      "id_especialidad": 1,
      "nombre": "Medicina General",
      "descripcion": "Atención médica general"
    },
    {
      "id_especialidad": 2,
      "nombre": "Pediatría",
      "descripcion": "Atención médica para niños"
    },
    {
      "id_especialidad": 3,
      "nombre": "Cardiología",
      "descripcion": "Atención de enfermedades del corazón"
    }
  ]
}
```

Impresión con sangría ☒

```
{
  "status": "ok",
  "data": [
    {
      "id_medico": 1,
      "id_especialidad": 1,
      "nombre": "Laura",
      "apellido_paterno": "Hernández",
      "cedula_profesional": "MED001",
      "correo": "laura.hernandez@medici.com",
      "telefono": "5511111111"
    },
    {
      "id_medico": 2,
      "id_especialidad": 2,
      "nombre": "Carlos",
      "apellido_paterno": "Ramírez",
      "cedula_profesional": "MED002",
      "correo": "carlos.ramirez@medici.com",
      "telefono": "5522222222"
    },
    {
      "id_medico": 3,
      "id_especialidad": 3,
      "nombre": "Ana",
      "apellido_paterno": "Martínez",
      "cedula_profesional": "MED003",
      "correo": "ana.martinez@medici.com",
      "telefono": "5533333333"
    }
  ]
}
```

Impresión con sangría ☒

```
{
  "status": "ok",
  "data": [
    {
      "id_consultorio": 1,
      "nombre": "Consultorio 1",
      "ubicacion": "Planta baja"
    },
    {
      "id_consultorio": 2,
      "nombre": "Consultorio 2",
      "ubicacion": "Planta baja"
    },
    {
      "id_consultorio": 3,
      "nombre": "Consultorio 3",
      "ubicacion": "Primer piso"
    }
  ]
}
```



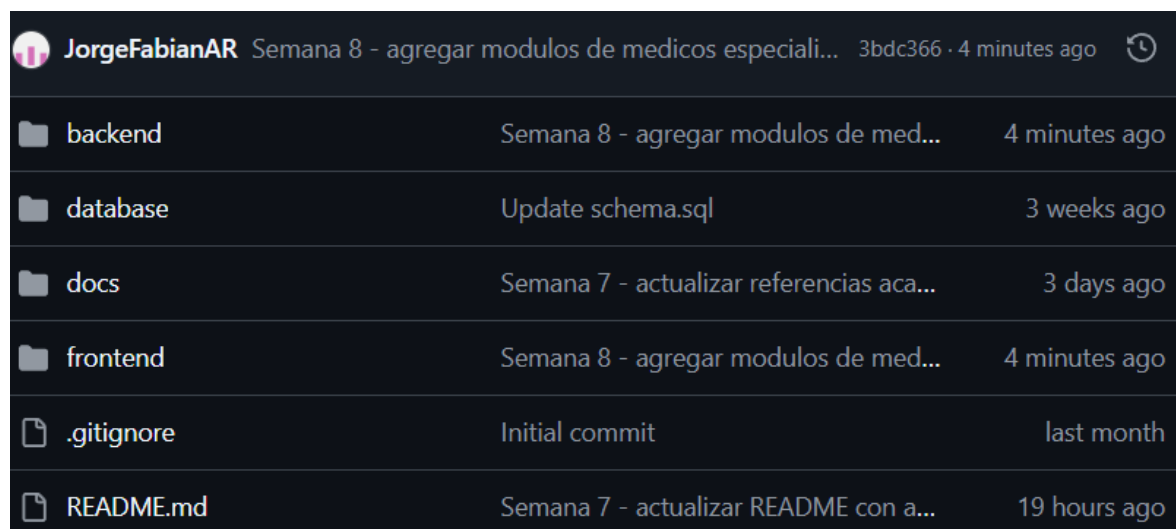
La evidencia de Semana 8 corresponde a las capturas de pantalla de los endpoints `/api/especialidades`, `/api/medicos`, `/api/consultorios` y del dashboard inicial del frontend. Estas evidencias se almacenan en el repositorio dentro de la carpeta `docs/evidencias_semana8`.

23.5 Relación con la hipótesis del proyecto

El avance de Semana 8 es necesario para preparar la implementación del módulo de citas médicas y la validación de disponibilidad. Para poder automatizar el agendamiento de citas, el sistema primero debe contar con datos de pacientes, médicos, especialidades y consultorios. Por ello, estos módulos son la base funcional que permitirá posteriormente medir tiempos de registro, errores de empalme, citas exitosas e intentos fallidos.

23.6 Commits realizados en Semana 8

Durante esta semana se realizó el commit correspondiente a la integración de módulos de médicos, especialidades y consultorios, así como la actualización del frontend con un dashboard inicial.



 JorgeFabianAR	Semana 8 - agregar modulos de medicos speciali...	3bdc366 · 4 minutes ago	
 backend	Semana 8 - agregar modulos de med...	4 minutes ago	
 database	Update schema.sql	3 weeks ago	
 docs	Semana 7 - actualizar referencias aca...	3 days ago	
 frontend	Semana 8 - agregar modulos de med...	4 minutes ago	
 .gitignore	Initial commit	last month	
 README.md	Semana 7 - actualizar README con a...	19 hours ago	

24. Atención a retroalimentación de Bitácora 1

Derivado de la retroalimentación recibida en la Bitácora 1, se identificaron áreas de mejora relacionadas con la calidad de las referencias bibliográficas, la evidencia de commits semanales y la trazabilidad del avance del MVP.

Como acción correctiva, se integraron fuentes académicas indexadas sobre sistemas de citas médicas, programación de citas en servicios de salud e interoperabilidad en entornos médicos. Asimismo, se corrigieron referencias registradas como “s.f.”, sustituyéndolas por fuentes con fecha de publicación, versión o actualización verificable.

También se reforzó la evidencia semanal del repositorio mediante commits descriptivos y capturas del historial de GitHub. A partir de Semana 7 se mantendrá al menos un commit significativo por semana, reflejando avances técnicos y documentales del sistema MediCI.

Estas acciones serán consideradas en el entregable final, con el objetivo de atender directamente las observaciones señaladas y fortalecer la fundamentación científica y técnica del proyecto.

25. Referencias

- Ala, A., Alsaadi, F. E., Ahmadi, M., & Mirjalili, S. (2022). *Appointment Scheduling Problem in Complexity Systems of the Healthcare Services: A Comprehensive Review*. Journal of Healthcare Engineering, 2022, 1-16. <https://doi.org/10.1155/2022/5819813>
- Bettoni, G. N., Camargo, T., Santos, B. G. T., Flores, C. D., & Silva, F. S. (2021). *Application of HL7 FHIR in a Microservice Architecture for Patient Navigation on Registration and Appointments*. arXiv. <https://arxiv.org/abs/2103.07589>
- HL7 International. (2026). *Appointment - FHIR v6.0.0-ballot4*. HL7 FHIR. <https://build.fhir.org/appointment.html>
- Meta Open Source. (2025). *Quick Start*. React. <https://react.dev/learn>
- Mozilla Developer Network. (2026). *JavaScript*. MDN Web Docs. <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- Node.js. (2026). *Node.js v24.18.0 (LTS)*. OpenJS Foundation. <https://nodejs.org/en/blog/release/v24.18.0>
- Open Worldwide Application Security Project. (2025). *OWASP Top Ten Web Application Security Risks*. OWASP Foundation. <https://owasp.org/www-project-top-ten/>
- Oracle. (2026). *MySQL Documentation*. Oracle. <https://dev.mysql.com/doc/>
- The Express.js Project. (2025). *Routing*. Express.js. <https://expressjs.com/en/guide/routing.html>
- World Health Organization. (2021). *Global strategy on digital health 2020-2025*. World Health Organization. <https://www.who.int/publications/i/item/9789240020924>
- Zhao, P., Yoo, I., Lavoie, J., Lavoie, B. J., & Simoes, E. (2017). *Web-Based Medical Appointment Systems: A Systematic Review*. Journal of Medical Internet Research, 19(4), e134. <https://doi.org/10.2196/jmir.6747>
-

26. Conclusión de avances Semana 6 a Semana 8

Con los avances realizados hasta Semana 8, el proyecto MediCI ha pasado de una fase de definición y arquitectura a una fase de desarrollo funcional inicial. En Semana 6 se consolidó el MVP esqueleto, demostrando la conexión entre frontend y backend. En Semana 7 se atendió la retroalimentación de la Bitácora 1 y se inició el módulo

funcional de pacientes. En Semana 8 se agregaron los módulos de especialidades, médicos y consultorios, además de un dashboard inicial en el frontend que consume datos desde la API.

Estos avances fortalecen la construcción del sistema y preparan la implementación del módulo de citas médicas, la validación de empalmes y el registro de logs, los cuales serán fundamentales para generar al menos 20 registros reales y validar la hipótesis del proyecto.

La actualización constante del repositorio de GitHub, los commits semanales, la documentación de evidencias y la incorporación de fuentes académicas permiten demostrar que el proyecto avanza conforme al protocolo y atiende las observaciones recibidas en la primera bitácora.